UNIDAD 1 - RECONOCIMIENTO DE ELEMENTOS DEL DESARROLLO DEL SOFTWARE

- UNIDAD 1 - RECONOCIMIENTO DE ELEMENTOS DEL DESARROLLO DEL SOFTWARE
  - 1. GLOSARIO
  - 2. INTRODUCCIÓN - Características del Software
    - Tipos de Software
  - 3. PROGRAMAS INFORMÁTICOS - Tipos
  - 4. LENGUAJES DE PROGRAMACIÓN
    - Tipos de Lenguajes de Programación
      - Clasificación de Lenguajes según su ejecución
  - 5. PROCESO DE TRADUCCIÓN / COMPILACIÓN
    - Intérprete
    - Compilador
    - PUNTOS CLAVE
  - 6. CLASIFICACIÓN DE LOS LENGUAJES DE PROGRAMACIÓN Y ÁMBITOS DE APLICACIÓN
  - 7. DESARROLLO DE SOFTWARE
    - Fases del desarrollo de un software
    - Documentación
    - Roles que forman parte del proceso de desarrollo

1. GLOSARIO

- **Bytecode**: Código interpretable directamente por una máquina virtual, como la JVM en Java.
- **Portabilidad**: Capacidad de un software para ser ejecutado en diferentes sistemas sin necesidad de modificaciones.
- **Firmware**: Software embebido en hardware que controla hardware específico. La BIOS de un ordenador es un ejemplo. Se almacena en memoria ROM o flash.
- **Instrucción**: Conjunto de datos insertados en una secuencia estructurada para que el procesador los interprete y ejecute.
- **Refactorizar código**: Proceso de optimización de código sin cambiar la lógica del mismo.
- **TypeScript**: Lenguaje de programación de código abierto. Es un superconjunto tipado de JavaScript que permite la detección temprana de errores relacionados con el tipo, haciendo que el código sea más fiable y menos propenso a errores.
- **FrameWork**: Conjunto de herramientas y bibliotecas que proporcionan soporte y facilita la programación. Proporciona una base completa sobre la que se puede empezar a construir el código. A diferencia de las librerías, el FrameWork es un marco de trabajo que brinda un set de herramientas para desarrollar sistemas o aplicaciones. Por lo general contiene librerías, pero no siempre. Ejemplos de FrameWorks:
    - AngularJs
    - Django
    - ASP.NET
- **Ejecutable**: Archivo que contiene un programa listo para ser ejecutado por el sistema operativo.
- **Paradigma**: Manera o estilo de programación. Existen diferentes paradigmas de la programación: Funcionales, Imperativos, Lógicos, Orientados a objetos, etc.
- **Clase** Plantilla o modelo a partir del cual se crean objetos en la Programación Orientada a Objetos (POO). Define atributos y métodos comunes para los objetos que se derivan de ella.
- **POO**: Programación Orientada a Objetos. Paradigma de programación basado en el concepto de "objetos", los cuales contienen en forma de campo unos atributos o variables (características) y el código en forma de métodos o funciones (comportamientos). El modelo a partir del cual se instancian los diferentes objetos del mismo tipo se le denomina "clase".
- **Servicio web**: Software intermediario que permiteque nuestra aplicación se comunique con otras aplicaciones a través de la web, utilizando protocolos estándar como HTTP y formatos de datos como XML o JSON.
- **Lenguaje Máquina**: Es el lenguaje nativo del ordenador, compuesto por instrucciones en código binario (0s y 1s). Es difícil de entender y escribir para los humanos. Las instrucciones no son portables entre diferentes arquitecturas de hardware, ya que cada tipo de procesador tiene su propio conjunto de instrucciones. Nadie programa directamente en lenguaje máquina hoy día.
- **Lenguaje de Alto Nivel**: Lenguajes de programación que están más cerca del lenguaje humano y son más fáciles de entender y escribir, suelen tener librerías y funciones predeterminadas y en ocasiones ofrecen frameworks para que la programación sea más intuitiva y eficiente. Suelen trabajar con orientación a objetos (POO). Ejemplos: Java, Python, C++, JavaScript.
- **Lenguaje de Bajo Nivel**: Lenguajes de programación que están más cerca del lenguaje máquina y son más difíciles de entender y escribir para los humanos. Permiten un mayor control sobre el hardware y la memoria, pero requieren un conocimiento más profundo de la arquitectura del sistema. Ejemplos: Ensamblador, C.

---

2. INTRODUCCIÓN - Características del Software

Las **Características principales del Software** son:

- **Es lógico, no físico. Es intangible**
- **Se desarrolla, no se fabrica**
- **No se estropea, pero puede volverse obsoleto**

Tipos de Software

1. **Software de Sistema**: Gestiona los recursos del hardware y proporciona servicios a otros tipos de software. Ejemplos: Sistemas Operativos (Windows, Linux), Controladores de dispositivos, Utilidades del sistema.
2. **Software de Aplicación**: Programas diseñados para realizar tareas específicas para el usuario. Ejemplos: Procesadores de texto (Microsoft Word), Hojas de cálculo (Excel), Navegadores web (Chrome, Firefox).
3. **Software de Programación/Desarrollo**: Son Entornos de Desarrollo (IDE), herramientas que los desarrolladores utilizan para crear, depurar, mantener o soportar otros programas y aplicaciones. Ejemplos: Editores de código (Visual Studio Code), Compiladores (GCC), Entornos de desarrollo integrados (IDEs como Eclipse, IntelliJ IDEA).

---

3. PROGRAMAS INFORMÁTICOS - Tipos

Un programa es una serie de órdenes o instrucciones secuenciadas para realizar una función o tarea determinada.

- **Software a medida**: Software desarrollado según los requerimientos de una empresa, organismo o usuario específico. El software se adapta a las actividades y necesidades del cliente. Es el tipo de software más caro, ya que se desarrolla desde cero y en ocasiones requiere de un equipo especializado y métodos no estandarizados. Ejemplos: ARCOnexo (Software interno de Logística en la Corporación del automóvil Jimenez Maña).
- **Software estándar**: Software genérico desarrollado para un público amplio y con funcionalidades comunes. Se diseña para ser adaptable a diferentes usuarios y situaciones, aunque no se adaptará perfectamente a las necesidades específicas de cada usuario, por ello suele ser la empresa o usuario el que se adapte al Software. Suele ser mucho más barato y se compra ya hecho. Ejemplos: Microsoft Office, Adobe Photoshop, navegadores web.

---

4. LENGUAJES DE PROGRAMACIÓN

Los lenguajes de programación se han creado para ser traducidos a un lenguaje que el ordenador pueda entender (código máquina). Hoy en día existen muchos lenguajes de programación, cada uno con sus propias características, ventajas y desventajas. Uno de los más populares y conocidos es **Java**, aunque otros como **Python** están ganando mucha popularidad.

Tipos de Lenguajes de Programación

- **Lenguaje Máquina**: Es el lenguaje nativo del ordenador, compuesto por instrucciones en código binario (0s y 1s). Es difícil de entender y escribir para los humanos. Las instrucciones no son portables entre diferentes arquitecturas de hardware, ya que cada tipo de procesador tiene su propio conjunto de instrucciones. Nadie programa directamente en lenguaje máquina hoy día.
- **Lenguaje de Alto Nivel**: Lenguajes de programación que están más cerca del lenguaje humano y son más fáciles de entender y escribir, suelen tener librerías y funciones predeterminadas y en ocasiones ofrecen frameworks para que la programación sea más intuitiva y eficiente. Suelen trabajar con orientación a objetos (POO). Ejemplos: Java, Python, C++, JavaScript.

Clasificación de Lenguajes según su ejecución

- **Lenguajes Compilados**: El código fuente se traduce completamente a código máquina antes de su ejecución mediante un programa llamado **compilador**. El resultado es un archivo ejecutable independiente del código fuente. Ejemplos: C, C++, Go.
- **Lenguajes Interpretados**: El código fuente se **traduce y ejecuta línea por línea en tiempo real** mediante un programa llamado **intérprete**. No se genera un archivo ejecutable independiente. Ejemplos: Python, JavaScript, Ruby.
- **Lenguajes Híbridos**: Combinan características de lenguajes compilados e interpretados. El código fuente se compila a un código intermedio (bytecode) que luego es interpretado o ejecutado por una máquina virtual. Ejemplos: Java (compilado a bytecode y ejecutado por la JVM), C# (compilado a bytecode y ejecutado por la CLR).

---

5. PROCESO DE TRADUCCIÓN / COMPILACIÓN

Los **Traductores** son programas que convierten un lenguaje de alto nivel a código máquina. Existen dos tipos principales: Intérpretes y Compiladores.

Intérprete

Traduce y ejecuta el código línea por línea en tiempo real. No genera un archivo ejecutable independiente. Ejemplos: Python, JavaScript.

Compilador

Traduce todo el código fuente a código máquina antes de su ejecución, generando un archivo ejecutable independiente. Sólo existe en un IDE. El código generado solo funcionará en una máquina con un hardware y software específico. Durante la fase de compilación existen dos subfases:

1. **Análisis léxico**: Se detectan las operaciones, caracteres de puntuación, etc.
2. **Análisis sintáctico/semántico**: Se comprueba que las instrucciones tienen sentido y se ajustan a las reglas del lenguaje.

PUNTOS CLAVE

Tener clara la diferencia entre:

- **Código fuente**: Código escrito en un lenguaje de alto nivel por el programador.
- **Código objeto**: Código intermedio generado por el compilador, que puede necesitar ser enlazado para crear un ejecutable.
- **Código ejecutable**: Código máquina listo para ser ejecutado por el sistema operativo.

Las extensiones de código ejecutable pueden ser `exe`, `.bat`, `.com` en Windows, `apk`, `.bin`, `.elf` en Linux y `apk` en Android.

El **LINKER** es un programa encargado de tomar todos los archivos objeto generados por el compilador (o ya presentes en una biblioteca), y combinarlos en un solo archivo ejecutable.

---

6. CLASIFICACIÓN DE LOS LENGUAJES DE PROGRAMACIÓN Y ÁMBITOS DE APLICACIÓN

Lenguaje	Tipo	Ámbito de Aplicación
C	Compilado	Desarrollo de sistemas
C++	Compilado	Desarrollo de aplicaciones y juegos.
C#	Compilado	Desarrollo de aplicaciones para Windows, móviles, juegos y aplicaciones web con (ASP.NET)
Java	Compilado	Aplicaciones móviles y de escritorio
Swift	Compilado	Desarrollo de aplicaciones IOS, macOS
Rust	Compilado	Desarrollo de sistema y aplicaciones web.
Python	Interpretado	Desarrollo web y aplicaciones móviles
JavaScript	Interpretado	Desarrollo web, para el front-end y back-end
Ruby	Interpretado	Desarrollo web.
PHP	Interpretado	Desarrollo Web del lado del servidor (Back-end)

7. DESARROLLO DE SOFTWARE

El desarrollo de software es el proceso de **diseñar, crear, probar y mantener aplicaciones, marcos u otros componentes de software**. Implica una serie de etapas que van desde la concepción inicial hasta la implementación y el mantenimiento continuo del software.

Fases del desarrollo de un software

Son: **Planificación/Fase Inicial → Análisis → Diseño → Implementación/Codificación → Pruebas/Testing → Explotación → Mantenimiento**

1. **PLANIFICACIÓN/FASE INICIAL**: Se define si es rentable o no. Se establecen las bases de cómo va a desarrollarse el resto de fases.
2. **ANÁLISIS**: Se analiza el problema. Consiste en recopilar, examinar y formular los requisitos del cliente y analizar cualquier restricción que pueda aplicarse. Hay que registrar las entrevistas con el cliente y documentarlas e incluso firmarlas con los requisitos de la aplicación.
3. **DISEÑO**: Se definen los requisitos técnicos de la arquitectura del software. La documentación debe ser más técnica. Se realizan dos documentos, uno genérico de una visión más general y otro detallado que profundice en los detalles técnicos.
4. **IMPLEMENTACIÓN/CODIFICACIÓN**: Se implementa el software en un lenguaje de programación adecuado. Se crea documentación muy detallada teniendo en cuenta que en un futuro el código debe ser mantenido por otros desarrolladores.
5. **PRUEBAS/TESTING**: Se verifica que el software cumple con los requisitos especificados y funciona correctamente. Pueden ser de dos tipos principalmente:
    1. **Funcionales**: Se centran en probar cómo interactúan los diferentes componentes del software entre sí. Se realizan con el cliente.
    2. **Estructurales**: Las realiza el equipo de desarrollo. Consiste en examinar el rendimiento y fluidez de la aplicación.
6. **EXPLOTACIÓN**: Instalación de software en el entorno real y se trabaja con él. Es la fase con más incidencias. Todas las incidencias se detallan en un documento que incluye errores o fallos que deben ser revisados. También pueden surgir otras necesidades que se realizarán en la siguiente fase.
7. **MANTENIMIENTO**: Se realizan procedimientos de corrección y actualización del software. Consiste en adaptar y evolucionar la aplicación. Es necesario de la documentación técnica sino las labores serán muy difíciles y su garantía de éxito será muy baja.

Documentación

La documentación es fundamental en todas las fases del desarrollo de software. **Sirve para comunicar ideas, requisitos, diseños y procedimientos entre los miembros del equipo y con los clientes.**

En cualquier aplicación debe generarse los siguientes documentos:

1. **Manual de Usuario**: **Manual que utilizará el usuario final para desenvolverse con el software**. Debe servir para aprender cómo se maneja la aplicación, sus funcionalidades y características principales. Debe estar redactado en un lenguaje sencillo y claro, evitando tecnicismos innecesarios.
2. **Manual Técnico**: **Manual dirigido a los desarrolladores y personal técnico que trabajará en el mantenimiento y actualización del software**. Debe incluir detalles sobre la arquitectura del software, tecnologías utilizadas, estructura del código, procedimientos de instalación, configuración y solución de problemas. Debe estar redactado en un lenguaje técnico adecuado para profesionales del área.
3. **Manual de Instalación**: **Manual que detalla paso a paso los requisitos y cómo debe de instalarse y poner en funcionamiento el software**. Debe incluir los requisitos del sistema, instrucciones de instalación, configuración inicial y solución de problemas comunes durante la instalación.

Roles que forman parte del proceso de desarrollo

El equipo de desarrollo de software puede incluir varios roles, los cuales tiene unas atribuciones y responsabilidades específicas:

- **Arquitecto de Software**: **Se encarga de cómo va a realizarse el proyecto**. Tiene conocimientos profundos de las tecnologías, frameworks, librerías, programación... Decide la forma y recursos con los que se lleva a cabo el proyecto.
- **Jefe de Proyecto**: **Es el que dirige**. Su función es saber gestionar el equipo y los tiempos, tener comunicación con el cliente y asegurarse de que el proyecto se realiza según lo planificado.
- **Analista de Sistemas**: **Se encarga de recopilar y analizar los requisitos del cliente**. Traduce las necesidades del cliente en especificaciones técnicas para el equipo de desarrollo. Se encarga de hacer el diseño del sistema.
- **Desarrollador/Programador**: **Es el que escribe el código**. Se encarga de implementar las funcionalidades del software según las especificaciones proporcionadas por el analista de sistemas.
- **Tester/QA (Aseguramiento de la Calidad)**: **Se encarga de probar el software para identificar errores, fallos y asegurar que cumple con los requisitos especificados**. Realiza pruebas funcionales y estructurales. En ocasiones el desarrollador también puede hacer las funciones de tester.

En proyectos pequeños, el propio programador realiza las tareas de arquitecto, analista y programador.
